Software Documentation: Full-Stack Contact Management Application

Project Scope: A web application for managing contacts with **CRUD** (Create, Read, Update, Delete), **Search**, **Sorting**, and **Categorization** features. **User authentication is excluded** from the initial scope.

1. Technology Stack & Prerequisites

Layer	Technology	Version	Purpose
Frontend	**React**	Latest (CRA/Vite)	User Interface and interaction.
Backend	**Java** (Spring Boot)	17+ / Latest	RESTful API and Business Logic.
Database	**SQL** (e.g., PostgreSQL, MySQL)	Latest	Persistent storage for contact data.
Tools	**Maven/Gradle**, **Node.js**, **VS Code/IntelliJ**	N/A	Build and Development environment.

Export to Sheets

2. Part 1: Backend Setup (Java/Spring Boot & SQL)

The backend handles data persistence and provides a RESTful API for the frontend.

2.1. Database Schema (The Data Model)

Create a SQL table to store contact information. This schema supports the required features (Name, Phone, Email, and Categorization).

SQL
```
CREATE TABLE contact (
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100),
    phone_number VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(150) UNIQUE,
```

```
    category VARCHAR(50) NOTt NULL DEFAULT 'Personal',
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Example categories: 'Personal', 'Work', 'Family', 'Other'
```

2.2. Spring Boot Project Structure

Use **Spring Initializr** to generate a Maven/Gradle project with the following core dependencies:

- **Spring Web:** For building RESTful APIs.
- **Spring Data JPA:** For database interaction (ORM).
- **Database Driver:** (e.g., `postgresql` or `mysql-connector-java`).
- **Lombok** (Optional but recommended for boilerplate reduction).

Key Components:

Model (`Contact.java`): JPA Entity mapping to the `contact` table.

```
Java
@Entity
public class Contact {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String firstName;
    private String lastName;
    private String phoneNumber;
    private String email;
    private String category; // Used for categorization feature
    // Getters and Setters (via Lombok or manual)
}
```

1.

Repository (`ContactRepository.java`): Interface for database access.

```
Java
public interface ContactRepository extends JpaRepository<Contact, Long> {
    // Custom method for the search feature
    @Query("SELECT c FROM Contact c WHERE LOWER(c.firstName) LIKE %?1% OR
LOWER(c.lastName) LIKE %?1% OR LOWER(c.phoneNumber) LIKE %?1%")
    List<Contact> searchContacts(String searchTerm);
}
```

2.

3. **Controller (`ContactController.java`):** Exposes the REST API endpoints.

2.3. RESTful API Endpoints (CRUD + Features)

The controller must implement the following endpoints to support all required features.

Feature	Method	Endpoint	Description
Read All	`GET`	`/api/contacts`	Retrieves all contacts.
Read One	`GET`	`/api/contacts/{id}`	Retrieves a single contact.
Create	`POST`	`/api/contacts`	Adds a new contact (Request Body: `Contact` object).
Update	`PUT`	`/api/contacts/{id}`	Updates an existing contact.
Delete	`DELETE`	`/api/contacts/{id}`	Removes a contact.
Search	`GET`	`/api/contacts/search?term=...`	Searches by name or phone (uses the custom Repository method).
Sort/Filter	`GET`	`/api/contacts?sort=lastName&category=Work`	Supports sorting and filtering via URL parameters (handled by Spring Data's `Pageable` or custom logic).

Export to Sheets

---

3. Part 2: Frontend Setup (React)

The frontend will manage the UI, user state, and communication with the Java API.

3.1. Project Setup

Create a React project using Vite or Create React App (CRA).

Bash
npx create-vite contact-app --template react
cd contact-app
npm install axios # For making API requests

3.2. Core Component Structure

The application should be structured with functional components and React Hooks (`useState`, `useEffect`).

- **`App.js`:** Main container, manages global state (e.g., the full list of contacts).
- **`ContactList.js`:** Displays the list of contacts, iterates over the state.
- **`ContactForm.js`:** Handles Create/Update operations.
- **`Header.js`:** Contains the **Search Bar** and **Sorting/Filtering** controls.
- **`ContactCard.js`:** Displays individual contact details.

3.3. API Service Layer

Create an API file (e.g., `src/services/contactService.js`) to centralize all API calls using **Axios**.

JavaScript
```
import axios from 'axios';

const API_URL = 'http://localhost:8080/api/contacts'; // Adjust port if necessary

export const getAllContacts = () => {
    return axios.get(API_URL);
};

export const createContact = (contact) => {
    return axios.post(API_URL, contact);
};

export const deleteContact = (id) => {
    return axios.delete(`${API_URL}/${id}`);
};

// Search Implementation
export const searchContacts = (term) => {
    return axios.get(`${API_URL}/search?term=${term}`);
};
```

---

4. Part 3: Feature Implementation

4.1. CRUD Functionality (Data Flow)

1. **Read:** On initial load (`App.js`), use `useEffect` to call `contactService.getAllContacts()`. Store the result in a state variable (e.g., `[contacts, setContacts] = useState([])`).
2. **Create/Update:** `ContactForm.js` handles form submission. Upon successful API response (`POST`/`PUT`), update the `contacts` state in `App.js` (by fetching the full list again or manipulating the state directly).
3. **Delete:** `ContactCard.js` handles the delete button. Call `contactService.deleteContact(id)` and then filter the `contacts` state to remove the deleted item instantly.

4.2. Searching Feature

The search is primarily handled by the **Backend API**.

1. **Frontend (`Header.js`):**
    - An input field tracks the search term state.
    - Use a **debounce** function (recommended) on the input's `onChange` event to prevent excessive API calls.
2. **Trigger:** When the debounced term changes, call `contactService.searchContacts(term)`.
3. **Update:** `App.js` receives the filtered list from the API and updates the main `contacts` state, causing `ContactList.js` to re-render with the search results.

4.3. Sorting and Categorization

These features can be handled on either the frontend or the backend.

Backend Approach (Recommended for large datasets):

1. Modify the `GET /api/contacts` endpoint to accept `sort` and `category` URL parameters (e.g., `/api/contacts?sort=lastName&category=Work`).
2. Use Spring Data JPA's `Sort` and custom query methods within the `ContactRepository` to fetch the filtered and sorted data directly from the SQL database.

Frontend Approach (Simpler for initial scope):

1. Fetch the full list of contacts.

In `ContactList.js`, apply the sorting or filtering logic **before** rendering the list.
JavaScript
// Example Sorting Logic in React:
const sortedContacts = [...contacts].sort((a, b) => {
    if (a.lastName < b.lastName) return -1;
    if (a.lastName > b.lastName) return 1;
    return 0;
});

// Example Filtering/Categorization Logic in React:

2. const filteredContacts = contacts.filter(contact => contact.category === selectedCategory);